

Mako Markets — Whitepaper

Short-form parimutuel prediction markets on Monad.

Version 1.0 · April 2026 · makomarket.xyz

1. Abstract

Mako Markets is an on-chain prediction-market protocol deployed on the Monad testnet. It lets anyone create a YES/NO market, bet MON on either side, and settle against a real-world outcome resolved by a permissioned resolver or a public grace-period fallback. Markets are **parimutuel**: the total losing pool (minus fees) is distributed pro rata to winners. Three market types are supported out of the box — football fixtures, crypto spot-price thresholds, and NBA games — each backed by an automated off-chain resolver that calls the contract. The design prioritises short duration (≤ 7 days), minimum viable liquidity guards, pull-based payouts, and a clear separation between protocol economics and governance.

2. Problem

Most on-chain prediction markets optimise for long-tail events and demand heavy capital and sophisticated order-book UX. Short-form markets (a single match, a 24-hour price question) are under-served: the UX is heavy, resolution requires bespoke oracles, and tiny pools produce dishonest outcomes or no outcome at all. Centralised sportsbooks solve the UX problem but put custody, settlement, and censorship risk on a single operator. There is a gap for **micro-duration, on-chain, parimutuel markets** that anyone can spin up in under a minute and trust to settle within 48 hours of close.

3. Solution

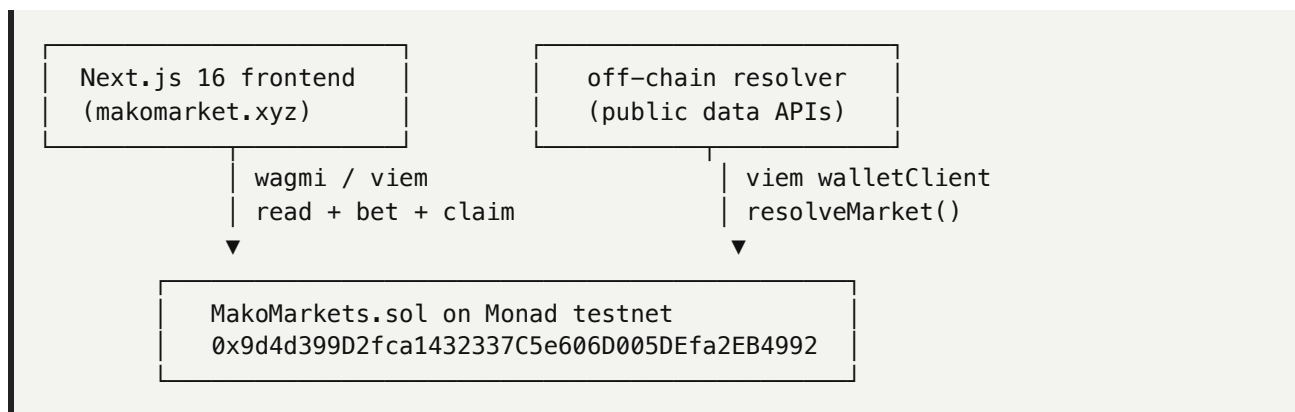
Mako Markets is a single smart contract plus a thin orchestration layer:

- **Contract** handles market creation, betting, resolution, payouts, and fee custody. No external price oracle is trusted on-chain; the contract only accepts a resolver-signed outcome.
- **Off-chain resolver** polls public market-data APIs (crypto spot prices, football match results, NBA box scores) and calls `resolveMarket(id, outcome)` when a market crosses `closeTime`.

- **Frontend** is a Next.js 16 app with wagmi v2 + viem + RainbowKit. It surfaces an explorer-style feed, a 3-tap create flow, a one-screen bet sheet, and an admin dashboard for the protocol operator.
- **Fallback** lets anyone call `forceRefund(id)` 24 hours after close if the resolver is offline — bettors get their stakes back without operator cooperation.

The whole stack ships as two repos: `mako-contracts` (Foundry) and `mako-markets` (Next.js). The live contract is at `0x9d4d399D2fca1432337C5e606D005DEfa2EB4992` on Monad testnet (chain id 10143).

4. Architecture



- **No proxies, no upgradeability:** a new version is a new deploy. Migrations are social, not storage. This keeps the audit surface tiny and the mental model simple.
- **No ERC-20:** the protocol takes native MON directly. One fewer approval transaction per bet, one fewer token contract in the trust graph.
- **No order book:** pools are parimutuel. There is no matching engine, no price-time priority, no adversarial MEV vector on placement.

5. Market Lifecycle

1. **Create.** A user calls `createMarket(mType, oracleRef, closeTime, question)`. `mType` is one of `FOOTBALL`, `CRYPTO`, `BASKETBALL`. `oracleRef` is a 32-byte encoding of the off-chain reference (e.g. `BTC:gt:77771` or `PL-Arsenal-ManCity-2026-04-21`). `closeTime` is constrained to `[now, now + 7 days]`. Question length ≤ 200 bytes.
2. **Bet.** While `block.timestamp < closeTime`, any wallet can `placeBet(id, isYes)` with `msg.value  $\geq$  0.001 MON`. Per-wallet YES and NO balances are tracked separately; unique-bettor counts are maintained per side. Bets are irrevocable.

3. **Close.** At `closeTime`, betting stops atomically. No transaction is required; the contract enforces the cutoff on the next `placeBet` attempt.
4. **Resolve.** The resolver calls `resolveMarket(id, outcome)` with `YES`, `NO`, or `REFUND`. The contract auto-promotes any non-`REFUND` outcome to `REFUND` if the pool is too lopsided (see §7). Fees are snapshotted at this point.
5. **Claim.** Winners call `claim(id)` at any time after resolution. Creator calls `claimCreatorFee(id)` once. Protocol fees accumulate in `treasuryBalance` and are withdrawn by the treasury address.
6. **Safety valve.** If the resolver never calls, `forceRefund(id)` is callable by anyone after `closeTime + 24h`, flipping the market to `REFUND` and unlocking claims.

6. Fee Model & Parimutuel Math

Let W be the winner pool, L the loser pool, and b an individual winning bet. Let $f = f_{\text{protocol}} + f_{\text{creator}}$. Live values on the deployed contract are 100 bps protocol + 200 bps creator = 300 bps (3%) total.

The payout pool is:

$$\text{payoutPool} = (W + L) * (1 - f)$$

An individual winner receives:

$$\text{payout}(b) = b * \text{payoutPool} / W$$

This means: - **Winners' edge** is scaled by $1 + L/W$, reduced by the fee. - **Losers** lose everything (that's the whole loser pool). - **Creator** collects $f_{\text{creator}} * (W + L)$ once, only if the outcome is not `REFUND`. - **Protocol** accrues $f_{\text{protocol}} * (W + L)$ to `treasuryBalance`.

Fees are capped by constant (`MAX_TOTAL_FEE_BPS = 500`, i.e. 5%). Owner can tune `f_protocol` and `f_creator` within that cap.

Refunds return every bettor's exact stake, no fee charged. Creator fee is **not** payable on refund.

7. Safety Mechanisms

Reentrancy

All state-mutating external functions that transfer value (`placeBet` , `claim` , `claimCreatorFee` , `withdrawTreasury`) use a lightweight `nonReentrant` modifier. Storage writes precede all external calls. Payouts use `call` with a boolean check; failures revert the whole tx.

Dust-attack defence

A creator with the creator-fee payout as a target could, in principle, self-bet a large winning side and a tiny losing side to collect the creator fee off their own capital. To make this unprofitable, `resolveMarket` force-refunds any outcome where:

```
min(totalYes, totalNo) * 10000 < max(totalYes, totalNo) * minLiquidityRatioBps()
```

`minLiquidityRatioBps()` is dynamic: it equals $2 * \text{creatorFeeBps} / (10000 - \text{creatorFeeBps})$, floored at 100 bps. With the live 2% creator fee, the loser side must be $\geq \sim 4.08\%$ of the winner side for the market to settle. Below that, everyone is refunded — the creator eats the round-trip gas but nothing else.

Grace-period force refund

If the resolver is offline or the oracle reference is unparseable, the market would be stuck. 24 hours after `closeTime` , `forceRefund(id)` is callable by any wallet and moves the market to `REFUND` . This bounds the worst-case “user funds locked” window to 24 h regardless of operator liveness.

Input constraints

- `closeTime > now` and `closeTime ≤ now + 7 days`
- `1 ≤ bytes(question) ≤ 200`
- `MIN_BET = 0.001 MON` on every `placeBet`
- `protocolFeeBps + creatorFeeBps ≤ 500`

Ownership separation

`owner` can rotate `resolver` and `treasury` , tune fees within the cap, and transfer ownership. `resolver` can only call `resolveMarket` . `treasury` can only call `withdrawTreasury` . This keeps the hot-wallet surface (the resolver bot) stripped of any ability to drain funds or change economics.

8. Admin Dashboard

The protocol operator runs an authenticated dashboard at `/admin/*` on makomarket.xyz:

- **Overview** — platform stats, user growth curve, daily active wallets, last 10 events.
- **Users** — every address that has bet or created a market, sortable by volume, bet count, creator fees earned, or recency.
- **Markets** — full market list with filter by state (open / pending resolve / resolved).
- **Activity** — last 200 on-chain events (bets, market creations, resolutions, claims, creator fees) with explorer links.
- **Resolve** — one-tap YES/NO/REFUND buttons for markets past close.

Access is gated by a **Sign-In with Ethereum** (EIP-4361) flow: the admin wallet signs a domain-bound, chainId-bound, single-use nonce, and the server issues a session that expires after 24 hours. Unauthenticated analytics requests are rejected before any RPC work, keeping infrastructure cost bounded regardless of who probes the endpoint.

9. Deployment

- **Contract:** `0x9d4d399D2fca1432337C5e606D005DEfa2EB4992` on Monad testnet
- **Chain ID:** `10143`
- **Explorer:** testnet.monadexplorer.com
- **Web app:** makomarket.xyz
- **Contract source:** github.com/jesterkeri/mako-contracts
- **App source:** github.com/jesterkeri/mako-markets

Build: Foundry for contracts, Next.js 16 with pnpm + Turbopack for the frontend.

10. Risks & Limitations

- **Testnet:** the current deployment is Monad testnet. Mainnet deployment requires an audit pass and a fee/treasury review.
- **Resolver trust:** the resolver is a single hot wallet. A malicious resolver can publish wrong outcomes. The grace-period `forceRefund` bounds the damage to “winners don’t get paid correctly for 24 h, then everyone gets refunded” — they do not bound outright theft of the pool. A multi-sig resolver or a challenge-window design is a natural follow-up.

- **Off-chain oracle fragility:** public data APIs can rate-limit, fail, or return unexpected shapes. The auto-resolver skips unparseable markets and logs them; the grace-period safety valve is the ultimate backstop.
- **Short max duration:** 7 days is a design choice, not a limit of the math. Long-tail markets are explicitly out of scope.
- **No order book / no AMM:** the parimutuel design means bettors cannot exit before close. Secondary markets are out of scope.

11. Roadmap

- **Always-on resolver.** Move the resolver loop to a scheduled serverless route, so resolution is independent of any operator machine.
 - **Resolver rotation UX.** Admin page to rotate the resolver wallet via `setResolver(address)` and trigger a SIWE secret rotation in one flow.
 - **More market types.** Tennis, esports, governance votes — each is a new enum value + a new resolver adapter.
 - **L2 or mainnet.** Pending audit, a Base or Monad mainnet deploy.
 - **Referral split.** Optional 10% of the creator fee routable to a referring wallet, enabling third-party distribution.
-